Ders 6 Çalışma Özeti

Ders 6 Çalışma Özeti

Genel Konular

- Asıl anlatım yaklaşık 00:03:49'da başlar.
- Geçen haftaların tekrarı
  - İş analizi: business requirements dokümanı ve use case modellemesi.
  - Fonksiyonel çözümleme: veri akış diyagramları, süreç tanım formları.
- Problem uzayının modellenmesi (nesneye yönelik çözümleme)
  - Amaç: Gerçek dünyadaki problemin doğru, anlaşılır ve sınanabilir bir yazılım kopyasını (replika) kurmak.
  - Problem uzayı ile yazılım uzayı mümkün olduğunca aynı kavramlardan oluşmalı.
- UML (Unified Modeling Language)
  - Analizden devreye almaya kadar her aşama için diyagram sunan tümleşik bir modelleme dili.
- Kavramsal sınıf diyagramı (conceptual class diagram, domain model)
  - Analiz modelini gösterir. Tasarımda yazılım sınıf diyagramına dönüşür.
- Kavramsal sınıfların bulunması
  - Kategori listesi, yayınlanmış modellerin uyarlanması, use case'lerdeki isim ve isim tamlamaları.
- Gereksiz aday sınıfların elenmesi.
- Betimleme (tanımlama, specification) sınıfı: Item ve ProductSpecification örneği.
- Bağlantıların (association) belirlenmesi
  - Fiillerden, çoğullamalar (multiplicity), okuma yönü.
- Öz niteliklerin (attribute) belirlenmesi
  - Görünürlük, türetilmiş öz nitelik, çoğullama.
- Sonuç: Market satış sistemi için kavramsal sınıf diyagramı.
- Analiz aşamasının sonu ve SRS dokümanı.

Hocanın Özellikle Vurguladığı Kısımlar

- Kavramsal modelde yazılım ayrıntısı olmaz.
  - GUI kütüphanesi, veritabanı erişim sınıfı, veri taşıma nesnesi gibi şeyler bu aşamada düşünülmez. Bunlar tasarımın işidir.
- Use case'lerdeki bütün isimler ve isim tamlamaları aday sınıftır.
  - Ama hepsi sınıf olursa model gereksiz büyür. Mutlaka eleme yapılmalıdır.
- Bir aday gerçekten sınıf mı diye şu sorular sorulur:
  - Bu kavramla ilgili veri saklayacak mıyım?
  - Farklı değerler alan özellikleri var mı?
  - Bu kavramdan birden fazla nesne türeyebilir mi?
  - Uygulamanın kapsamında mı?
  - Sınırları iyi çizilmiş mi, net tanımlanabiliyor mu?
- Bağlantılar mümkün olduğunca ikili (iki sınıf arası) olmalıdır.
  - "Memur hesapla ilgili işlemleri girer" gibi üçlü ilişkiler kafa karıştırır. İkili bağlantılara bölün.
- Her fiil bir bağlantı değildir.
  - Hepsini çizerseniz diyagram okunmaz hâle gelir. Gereksiz bağlantıları eleyin.
- Birden fazla alanı olan, üzerinde işlem yapılan bir öz nitelik aslında ayrı bir sınıftır.
  - Kasiyerin "mevcut kasası" bir öz nitelik değil, ayrı bir Register (kasa) sınıfıdır. Fiyat

için kullanılan “para” da birim ve kur içeren ayrı bir kavram olabilir.
- Kavramsal model iteratif olarak gelişir.
  - İlk modelin yanlış olması gerekmez. Ama deneyimle daha iyi bir modele geçilir (betimleme sınıfı örneği).
- SRS dokümanı örneklerini mutlaka inceleyin.

Kısa Tekrar Notları

- Kavramsal sınıf diyagramında her kutunun iki bölmesi var: sınıf adı ve öz nitelikler.
  - Metotlar (üçüncü bölme) tasarımda eklenir.
- Kavramsal sınıf bulma yolları:
  - Kategori listesi: sık kullanılan kavramlar (öğrenci, ders, bölüm, transkript…). Özel problemlerde kavram kaçırma riski var.
  - Yayınlanmış modellerin uyarlanması: elde önceki sürümün modeli varsa.
  - Use case metinlerindeki isimler ve isim tamlamaları: her projeye uygulanabilir.
- Elenen aday sınıf türleri:
  - Fazlalık (redundant): aynı kavram için birden fazla kelime (good, item, product).
  - İlgisiz: o senaryoyla ilgisi olmayan (satış senaryosunda kredi kartı).
  - Belirsiz: sınırları çizilmemiş (muhasebe sistemi).
  - Öz nitelik: tek başına anlamı olmayan (barkod numarası, birim fiyat).
  - İşlem: fiil olanlar; davranış olarak modellenir.
  - Rol: bir ilişkideki rol (myCourses); sınıf değildir.
  - Yazılım sınıfları: adaptör, buton listesi gibi; tasarıma aittir.
- Çoğullamalar:
  - `1`: tam bir.
  - `0..1`: en çok bir.
  - * (`0..*`): sıfır veya daha fazla.
  - `1..*`: en az bir.
  - `1..3`: bir ile üç arası.
- Bağlantı adı fiil olur. Gerekirse okuma yönü okla belirtilir (“Öğrenci ders alır”, “Ders öğrenci alır” değil).
- Öz nitelik gösterimi:
  - `+` public.
  - `-` private.
  - `/` türetilmiş (derived).
  - `{readOnly}` salt okunur.
  - `[0..1]` gibi çoğullama.
- Analizin iki aşaması:
  - İş analizi: use case modeli.
  - Çözümleme: VAD (fonksiyonel) veya kavramsal sınıf diyagramı (nesneye yönelik).
  - Sonuç SRS dokümanına yazılır.

Detaylı Açıklamalar

Kavramsal modelden yazılım modeline

- Öğrenci bilgi sistemi örneği:
  - Kavramsal model: `Öğrenci (ad)` — alır — `Ders (kod)`. Bir öğrenci n tane ders alabilir.
  - Tasarımda yazılım sınıflarına dönüşür:
    - `Student`: `name: String`, `calculateAverage(): double`, `myCourses: List<Course>`.
    - `Course`: `number: String`, `getGrade(): double`.
- Kodda da `public class Student` ve `public class Course` yazılır. Yani kavramsal model, tasarımın ve kodun temelidir.

Use case’den aday sınıf çıkarma (market satış senaryosu)

- Senaryo kısaca (hoca senaryoda 10 adım olduğunu söyledi, 00:26:51; aşağıdaki liste özettir, numaraları slaytla birebir değildir):
  1. Müşteri kasaya ürünlerle gelir.
  2. Kasiyer yeni satış başlatır.

3. Kasiyer ürün kimliğini girer.
4. Sistem satış satırını kaydeder; ürün tanımını, fiyatı ve toplamı gösterir. Fiyat, fiyat kurallarıyla hesaplanır.
5. Vergiler hesaplanır.
6. Müşteri öder. Bu adım slaytta 7. adımdır (00:28:44). Alternatif akış 7a: Nakit ödemede verilen miktar girilir, sistem para üstünü gösterir.
7. Satış bilgileri muhasebe ve envanter sistemlerine gönderilir.
8. Fiş verilir.

- İşaretlenen isimler: customer, checkout, goods, cashier, sale, item identifier, sale line item, item description, price, price rules, total, taxes, payment, inventory, accounting, receipt.
- Eleme:
  - Ürün tanımı, ürün kimliği ve fiyat ayrı kavram değildir; ürünün öz nitelikleridir.
  - good ve item aynı kavramdır.

Betimleme (specification) sınıfı

- İlk model: `Item (description, price, serial number, itemID)`. Bu model yanlış değildir.
- Sorun: Mağazadaki 100 gofretin hepsi satılınca hiç `Item` nesnesi kalmaz. Gofretin tanımı, fiyatı ve barkodu da kaybolur.
- Çözüm: `ProductSpecification (description, price, itemID)` — Describes — `Item (serial number)`.
  - Tanım bilgisi ürünler satılsa da kalır.
  - Partiye göre değişen bilgiler (ör. son kullanma tarihi) `Item` nesnesinde tutulur.
- Bu sınıf gerçek dünyada yoktur. Model biraz gerçek dünyadan uzaklaşır ama daha iyi bir tasarıma yol açar.

Bağlantı örnekleri

- **Mağaza:**
  - Store 1 — Contains — 1..* Register.
    - Tasarımda `Store` içinde bir `registerList` olacağı daha şimdiden görülür.
  - Register 1 — 1..* Sale.
  - Sale 1 — Paid-by — 1 Payment.
    - Analize göre her satış tek ödemeyle yapılır. Parçalı ödeme olsaydı `1..*` ya da `1..3` yazılırdı.
- **Havayolu:**
  - Airline 1 — Employs — 1..* Person.
  - Person 1 — Supervises — * Person.
    - Kendine dönen bağlantı. “Yönetici” bir roldür, ayrı sınıf değildir.
    - Tasarımda `Person` içinde `supervisedBy: Person` alanı olur.
  - Person 1..* — Assigned-to — * Flight.
  - Plane 1 — Assigned-to — * Flight.
    - Okuma yönü okla belirtilir: Bir uçak birden çok uçuşa atanır.
- **Yaygın bağlantı türleri:** Fiziksel ya da mantıksal içerme, kaydetme, kullanma, tanımlama, sahip olma.
- **Bağlantı olarak gösterilmeyenler:**
  - Elenmiş sınıflara giden bağlantılar.
  - Senaryo dışı bağlantılar.
  - Gerçekleştirmeye ait bağlantılar.
  - Faaliyetler. Örnek: “ATM kredi kartını kabul eder” bir etkileşimdir, tasarımda gösterilir.

Market satış sistemi kavramsal sınıf diyagramı (slayttan)

Bağlantı	Çoğullama
Store — Houses — Register	1 — 1..*
Store — Stocks — Item	1 — *

Bağlantı	Çoğullama
ProductCatalog — Used-by — Store	1 — *
ProductCatalog — Contains — ProductSpecification	1 — 1..*
ProductSpecification — Describes — Item	1 — *
Register — Captured-on — Sale	1 — 0..1
Cashier — Works-on — Register	1 — 1
Sale — Contained-in — SalesLineItem	1 — 1..*
Sale — Paid-by — Payment	1 — 1
Sale — Is-for — Customer	1 — 1
Ledger — Records-accounts-for — Store	1 — 1
Ledger — Logs-completed — Sale	1 — *
SalesLineItem — Records-sale-of — Item	0..1 — 1..*

- Ürün kataloğu ayrı bir sınıf olduğu için birden fazla mağazada kullanılabilir (yeniden kullanılabilirlik).
- Son hâlde öz nitelikler de eklenir:
  - `ProductSpecification (itemID, description, price)`, `Item (serialNumber)`.
  - `Store (name, address)`, `Register (ID)`, `Cashier (ID, name: String)`.
  - Bazı öz nitelikler türetilmiştir (`/` ile).

Öz nitelikler

- Öz nitelik (attribute): Nesneye özgü değer alan veri. Basit veri tipleriyle ifade edilmelidir.
- Bir öz nitelik birden fazla alandan oluşuyorsa, üzerinde işlem yapılıyorsa ya da kendi öz nitelikleri varsa ayrı bir sınıf olmalıdır.
  - Örnek: Kredi kartının numarası ve geçerliliği doğrulanır.
- **Gösterim örnekleri:**
  - `+ pi: Real = 3.14 {readOnly}`: herkese açık, sabit.
  - `- x`: private.
  - `/ y`: başka niteliklerden hesaplanan türetilmiş değer.
  - `middleName [0..1]`: isteğe bağlı ikinci ad.
- Bu ayrıntılar kavramsal model için zorunlu değildir. Tasarıma özellikle taşımak istediğiniz noktalarda eklenir. Not (açıklama) kutusu da eklenebilir.

Analizden tasarıma

- Use case modeli (senaryolar) → domain model (kavramsal sınıflar).
- Tasarımda nesneler iş birliği yapar. Örnek:
  1. `Register.enterItem(id, quantity)` çağrılır.
  2. `Register`, katalogdan `ProductSpecification` ister.
  3. `Register`, `Sale.makeLineItem(spec, quantity)` ile satış satırı oluşturur.
- Bu konu ilerleyen haftalarda işlenecek.
- SRS dokümanı içerir:
  - Fonksiyonel çözümleme için: VAD'ler, süreç tanım formları, sözde kod ve karar tabloları.
  - Nesneye yönelik çözümleme için: senaryolar ve kavramsal sınıf diyagramı.

Ders sonrası notlar

- **Türetilmiş öz nitelik ile görünürlük karıştı:** Hoca "bir bölü görüyorsanız bu bir private özellik" dedi (01:14:10).
  - UML'de `/` yalnızca türetilmiş (derived) değeri gösterir. Görünürlükle ilgisi yoktur.
  - Görünürlük işaretleri: `+` public, `-` private, `#` protected, `~` package.
  - Türetilmiş bir öz nitelik public de olabilir.
- **`{readOnly}`:** Hoca buna "stereotip" dedi (01:13:39). UML'de stereotip «...» ile yazılır; `{readOnly}` bir özellik belirtecidir (property modifier).
- **Seri numarası:** Hoca katalogdan dönen bilgileri sayarken seri numarasını da andı (01:22:57). Diyagramda seri numarası `ProductSpecification`'da değil, `Item` üzerindedir.

- **Register–Sale çoğullaması iki slaytta farklı:**
  - Bağlantı örnekleri slaytında Register 1 — 1..* Sale var. Hoca bunu “sıfır satışlı kasa olmuyormuş” diye açıkladı (00:59:22).
  - Son diyagramda ise “Captured-on” bağlantısı 1 — 0..1. Bu, kasanın o anda işlediği tek satışı gösterir.
  - İkisi farklı analiz kararlarıdır.
- **SalesLineItem – Item okuması:** Bu slayttaki “Records-sale-of” bağlantısı satış kalemini fiziksel ürünlere (`Item`) bağlıyor: bir satış kalemi 1..* ürün kaydeder, her ürün en çok bir satış kaleminde yer alır. Hoca bağlantıyı okurken “item tanımı” dedi (01:07:30); oysa tanım bilgisi `ProductSpecification`’dadır.
  - Larman’ın kitabının bazı baskılarında bu bağlantı `SalesLineItem * – 1 ProductSpecification` olarak verilir; kaç adet satıldığı `quantity` öz niteliğinde tutulur. Başka kaynaklarda bu farklı gösterimi görürseniz şaşırmayın.
- **“Ödeme” bir işlem mi, kavram mı?** Hoca işlemlerin sınıf olmaması gerektiğini anlatırken ödemeyi örnek verdi. Ama diyagramda `Payment` bir sınıftır.
  - Ayrım şöyle: Bir varlık olarak “ödeme” (tutarı, türü olan kayıt) kavramsal sınıftır.
  - “Ödemeyi kabul et”, “para üstünü hesapla” gibi fiiller ise davranıştır. Bunlar tasarımda metot olur.
- **Altyazıdaki terimler:**
  - “Read-on” → `{readOnly}`.
  - “Derive detribüt” → derived attribute.
  - “GÖML” → UML.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.